

Distributed Storage Lab - Week 2

Written by: Marcell Feher <sw0rdf1sh@ece.au.dk>

Prerequisites

- You have completed Lab 1
- You are familiar with the material of Lecture 2
- You have a basic understanding of the request-response nature of HTTP
- You are familiar with the Javascript Object Notation ([JSON](#)) data format

Goals

In this lab, we will learn how to use JSON-RPC and how to provide a REST API in Python. For remote procedure calls, we will use the small and flexible library **tinyrpc**, which supports multiple transport methods including TCP/IP, HTTP and others. Then we will use the **Flask** library to implement a REST API, which is used very widely in modern application development. To test our simple file storage API, we are going to use a general purpose REST client called Postman.

JSON-RPC

Remote Procedure Call is a very generic concept of running a piece of code on a different machine, and receiving the result. It does not specify anything about how to actually implement this: what serialization format to use, how to structure the information that describes the call and the result, what transport method should carry the bits between the two parties, how to handle errors, and so on. Therefore, several solutions exist that address these problems in different ways. *Network File System* (NFS) for example uses predefined request and response types with fixed parameters, replies and errors. A different approach, *Simple Object Access Protocol* (SOAP) uses an XML-based self-descriptive format to define the whole interface provided by a server.

In this lab we will first use the **JSON-RPC**¹ protocol, which uses JSON to encode both the request (what function should be called with what parameters), and the result. It is flexible like SOAP because of the very general data format, but much more efficient on the wire due to the small overhead of JSON compared to XML.

For example, this is a JSON-RPC request:

¹ <https://www.jsonrpc.org/specification>

```
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [42, 23],
  "id": 1
}
```

As you probably figured, it requests the server to call `subtract(42, 23)`. Passing an ID means the caller wants to receive the result of the function. The result is encoded as the following JSON object:

```
{
  "jsonrpc": "2.0",
  "result": 19,
  "id": 1
}
```

The method parameters can either be encoded as an array, like the example above, or a JSON object where a name is assigned to each parameter value. Named parameters make the call easier to understand and maintain, but you are sending more bytes on the wire.

```
{
  "jsonrpc": "2.0",
  "method": "log_message",
  "params": {
    "message": "This is a sample error",
    "severity": "ERROR",
    "target": "file"
  },
  "id": 2
}
```

If the request represents a notification-like event that you don't need an answer for, just omit the ID field. However, the ID value is not only useful if there is a result that you are interested in, but also allows JSON-RPC to support **batch requests**. You can simply make an array of the request objects (each with different ID), and send them in a single request to the server. As a result you receive an array of response objects with the same IDs. It's okay to mix requests with

and without ID (for example when sending a combination of tasks and of notifications at the same time) and you can also send just an array of notifications.

Batch request:

```
[
  { "jsonrpc": "2.0", "method": "subtract", "params": [42, 23], "id": 1 },
  { "jsonrpc": "2.0", "method": "add", "params": [37, 3], "id": 2 },
  { "jsonrpc": "2.0", "method": "log_message", "params": { "message": "Log this message
using default severity and target" } },
  { "jsonrpc": "2.0", "method": "add", "params": [1, 1], "id": 3 }
]
```

Batch response:

```
[
  { "jsonrpc": "2.0", "result": 19, "id": 1 },
  { "jsonrpc": "2.0", "result": 40, "id": 2 },
  { "jsonrpc": "2.0", "result": 2, "id": 3 }
]
```

The result of the called Python function must be something that can be [encoded to JSON](#). Primitive types like numbers, strings, booleans and None are trivially encoded as the value of the **"result"** key in the response. Python lists and dictionaries become JSON arrays and objects. Everything else, for example an object of a custom Python class will trigger the *"TypeError: Object of type ... is not JSON serializable"* exception at the server (in this case you need to write a serializer to a JSON-supported type, and call it manually).

A nice addition of the TinyRPC library is that in case an exception is raised during the execution of a remote function, the error message is returned to the caller (see sample error response below). This way the caller understands what the problem is, instead of just having the connection closed without a response. Therefore when using TinyRPC, the rather unusual advice is to not handle exceptions yourself in the server code, but let them leave your function and get handled by the library. (Note: it's okay to catch exceptions, log and re-raise them)

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "message": "empty separator",
    "code": -32000
  }
}
```

```
}  
}
```

Unfortunately, if the exception occurs during parsing the JSON-RPC request or serializing the response (before or after your remote callable function, e.g. when a non-serializable Python object is returned), the server just hangs and nothing is returned to the caller.

Task 1: Run the HTTP-based quickstart example from the `tinypc` website

Go to the official [TinyRPC website](#) and locate the HTTP-based quickstart server and client codes. Save them on your computer as `rpc-server.py` and `rpc-client.py`, and run both with Python. You should see the following output:

```
$ python rpc-server.py  
127.0.0.1 - - [2020-08-31 18:25:33] "POST / HTTP/1.1" 200 340 0.000676  
  
$ python rpc-client.py  
Server answered: !dlroW ,olleH
```

Task 2: Read and try to understand both the server and client codes!

As you remember, RPC does not specify the transport protocol and neither does JSON-RPC. TinyRPC, unlike other libraries, actually supports multiple ways to transport messages between the client and server. You can choose between HTTP, a message queue called ZeroMQ (we'll learn it next week), websockets and a few other protocols out of the box, or you can implement your own. You can even use a different serialization format than JSON, called [MessagePack](#). Most of the server and client code are just selecting the transport protocol and setting up event handlers.

TinyRPC does a great job hiding the pain of serializing and parsing JSON manually. In the server all you need to do is annotate the remotely callable functions with `@dispatcher.public`

```
@dispatcher.public  
def reverse_string(s):  
    return s[::-1]
```

The RPC Client allows you to call remote functions just as they were local, using a proxy object:

```
remote_server = rpc_client.get_proxy()  
result = remote_server.reverse_string('Hello, World!')
```

As an additional bonus, since the server is listening on HTTP, you can even test it with Postman on your local computer!

Task 3: Call the server from Postman and reverse a string!

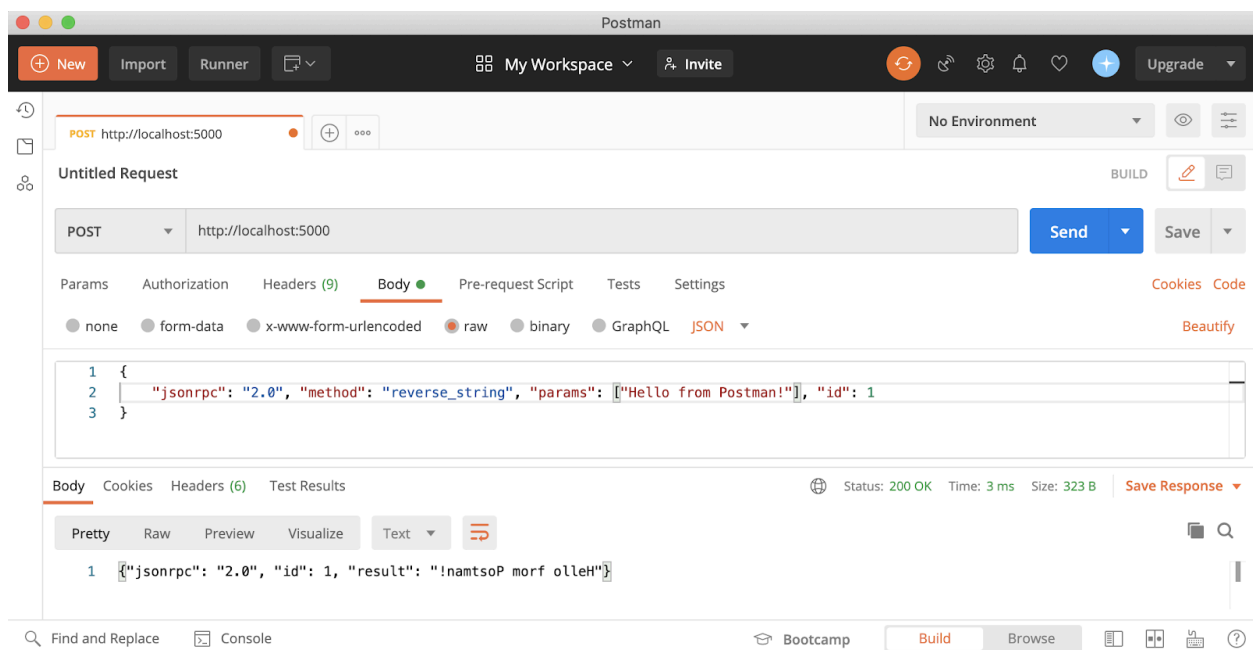
Open the Postman application and make a new request with the following settings:

Request type: POST

URL: <http://localhost:5000>

Body: "raw" data type, switch the syntax highlighter to JSON and add the following body:

```
{
  "jsonrpc": "2.0",
  "method": "reverse_string",
  "params": ["Hello from Postman!"],
  "id": 1
}
```



Binary data in JSON-RPC

While JSON-RPC is great for calling remote functions and delivering their result, it inherits the limitations of JSON. In distributed storage we regularly need to transport large chunks of binary data, for example when uploading a file from a client to a server, or when distributing it with added redundancy within the storage system itself. Unfortunately, JSON does not support raw, unstructured binary data. The usual way to transport binary data is to Base64 encode them into a string, but due to how the encoder works, this results in a 33% longer representation than the original binary data. If file transport happens rarely in your application and the files are not too

large (e.g. user profile picture), this is a totally acceptable solution. Otherwise more efficient serialization is necessary, like the custom binary protocol we used last week, or MessagePack, Protocol Buffers (see next week), multipart HTTP requests, or something similar.

Task 4: Extend `rpc-server.py` with a new remote callable function that accepts a file name (string) and contents (base64 string), and stores the file in the local file system. Test the new function with Postman!

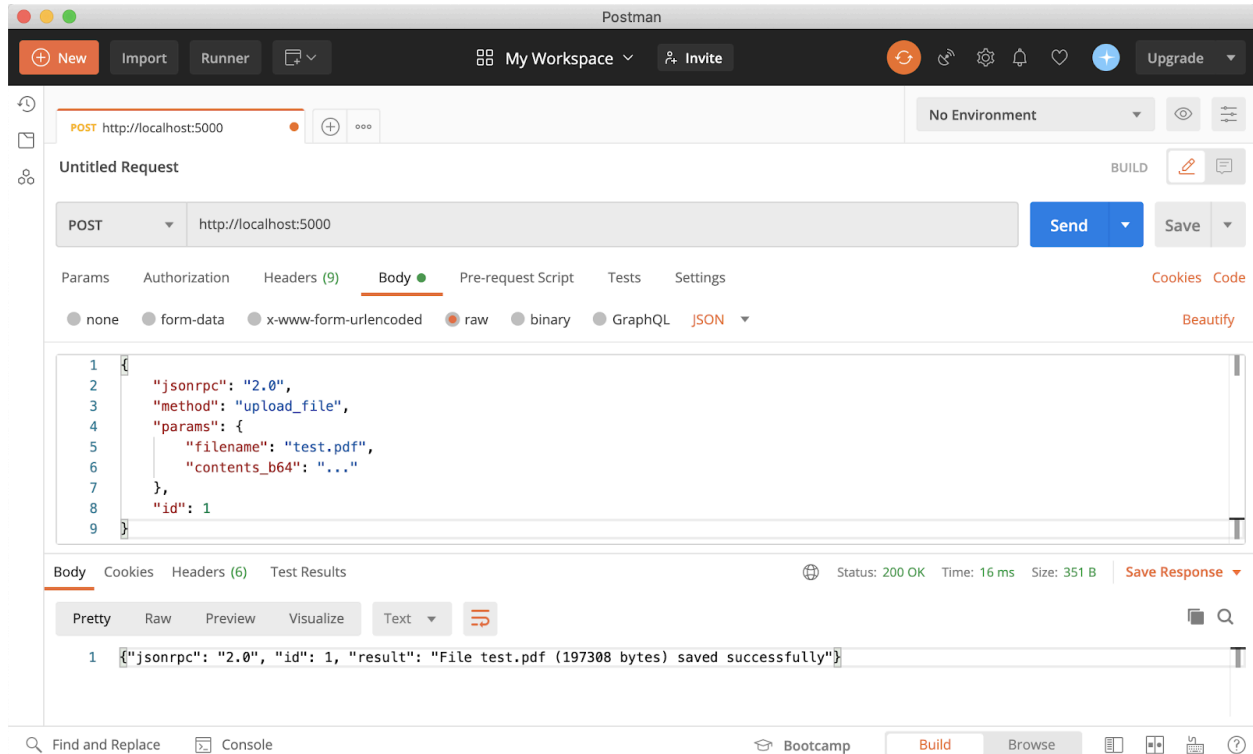
Python supports base64 encoding and decoding by default, the functions are located in the **base64** package. Usage:

```
import base64
...
binary_data = base64.b64decode(base64_string)
base64_string = base64.b64encode(binary_data)
```

You can re-use the `write_file()` Python function from last week to store the received and decoded file on the local file system. Don't forget to restart the server after adding the new endpoint.

To test the new function, find a non-sensitive, small (<50kb) file on your computer and convert it to base64 string using a [web-based converter](#). Then, update the Postman request body with the new function name and parameters, and send the request. You should see the file popping up in the same directory where `rpc-server.py` is. Verify that it has been decoded correctly, and the file opens without any problem.

For example:



RPC is a great choice when you are the one implementing both the client and the server. This is the case for example when you design a new distributed system and need a fast, low overhead protocol for the nodes to communicate between each other. However, it is not the standard way to provide a public interface to the outside world, designed for applications made by others. For that, we usually choose a REST API over HTTP, which we will discuss in the next section.

REST API with Flask

Representational State Transfer (REST) is a blueprint to design web APIs with a well-defined URL structure and use HTTP request types and status codes to carry information. It's similar to RPC since both allow the client to perform an action on a remote server, but REST does it in a declarative way. While in RPC you explicitly encode which function should be called with what parameters, a REST call encapsulates what action should be performed on what resource, and doesn't care about how the server does it.

There are three components of the REST design: the URL structure, HTTP request types and HTTP status codes. Let's briefly discuss each of these.

URL Structure

A REST API is a collection of URLs that a client can call via HTTP. It has a well-defined and constant *base URL* that usually ends with the version number, like <https://www.googleapis.com/drive/v3> for the latest version of the [Google Drive API](#).

The version number at the end allows the service provider to roll out a new version without affecting everyone using the older ones. After the base URL are the name and ID of the resource you want to do something with, for example `/user/123`. The request type defines what kind of operation you want to perform, and the request body can carry additional information (like the attributes to modify).

HTTP Request Types

There are four (+1) request types that are assigned to each resource type:

- GET request: Retrieve information
- POST request: Create a new instance of the resource
- PUT request: Edit an existing instance
- DELETE request: Remove an existing instance
- HEAD request: retrieve metadata about an instance, but do not return the instance itself. Used mostly for file attributes without returning the actual file contents.

HTTP Status Codes

A part of every HTTP response is a numeric, three-digit status code (also known as *response code*). A REST API uses this code to provide information to the caller about the action that was performed. The first digit of the status code determines the general result of the operation, while further digits give extra information:

- 2xx codes indicate success, like the general OK response (200) or resource created successfully (201)
- 4xx codes mean there is a problem with the request, e.g. invalid parameters (400), missing or wrong authorization (401) or the resource does not exist (404)
- 5xx codes indicate an error in the server itself that cannot be resolved by the client, for example an exception was raised during execution (500)

Putting these all together, a REST API consists of a set of resources and actions that can be performed on them. For example, consider the following endpoints of an imaginary object storage service.

User endpoints:

GET /user: List all users

GET /user/[id]: Return the details of one specific user

POST /user: Create a new user from the JSON object sent in the request body

PUT /user/[id]: Update the user who has the given ID, the changed attributes are in the request body as a JSON object

DELETE /user/[id]: Remove the given user

Container and object endpoints:

GET /container: List all containers

GET /container/[container-id]: List the contents (objects) of the given container

GET /container/[container-id]/[object-id]: Download the given object

HEAD /container/[container-id]/[object-id]: Get metadata of an object

...

Note

There is a specification and web-based editor to define a REST API, called [OpenAPI](#). It is highly recommended to use it when you are documenting an API.

For example, the OpenStack Object Storage API employs the REST concept in a really clean and easy to understand way, that is similar to our small example.

Task 5: Explore the [OpenStack Object Storage API](#) and identify how the REST principles are used.

Real life APIs can get a lot more complicated than pure REST and there are usually many additional endpoints that perform tasks that cannot be directly expressed by the four basic actions. Sometimes, the interface of a service consists of a combination of REST and other API types. Dropbox for example has two base URLs: **api.dropboxapi.com** for REST endpoints that manipulate metadata, and **content.dropboxapi.com** for uploading and downloading file contents. Keep in mind that REST is not a hard protocol that you either fully follow or don't use at all. It's just a set of best practices that can guide you when designing an API, and will be familiar to your users.

Note

Sometimes pure REST is unable to support the complete functionality you want to provide. GET and DELETE requests cannot have a request body and every parameter must be encoded in the URL itself. However, the length of the URL is limited to 2048 characters, which is not always enough.

For example, if you want to provide search/filter functionality in your list (GET) endpoint, the encoded query parameters can easily exceed this limit. Similarly, if you want to implement batch deleting resources where the list of IDs are passed to a DELETE request, it's possible that the end of this list is silently cut off before your code is invoked. In the best case you just failed to delete some resources, but if the last ID was cut in half, then you'll delete something that was not supposed to be removed!

In both of these cases the solution is to switch to a different HTTP request type that allows request body (POST or PUT), and diverge from the REST principles.

Flask web development framework

There are several good tools and frameworks to build web applications in Python, but two are dominating this field. [Django](#) is a fully featured framework that has a huge API and covers every

aspect of the development process. It takes some time to learn it, but you get a great tool to build and maintain large web platforms. For projects of smaller magnitude, the number one tool is the [Flask](#) microframework, which we will use in this course. It has a small API that is quick to learn but powerful enough to handle everything we need to implement a basic REST API. Flask also includes a template engine that helps if you want to produce HTML on the server and return it to the browser, but we will keep our interface JSON-only.

Task 6: Set up a basic Flask web application with a single endpoint, and test it from Postman!

Note

This task starts building a REST server that we will iteratively extend in the next 7 weeks.

Create a new file called **rest-server.py** and add the following content:

```
from flask import Flask

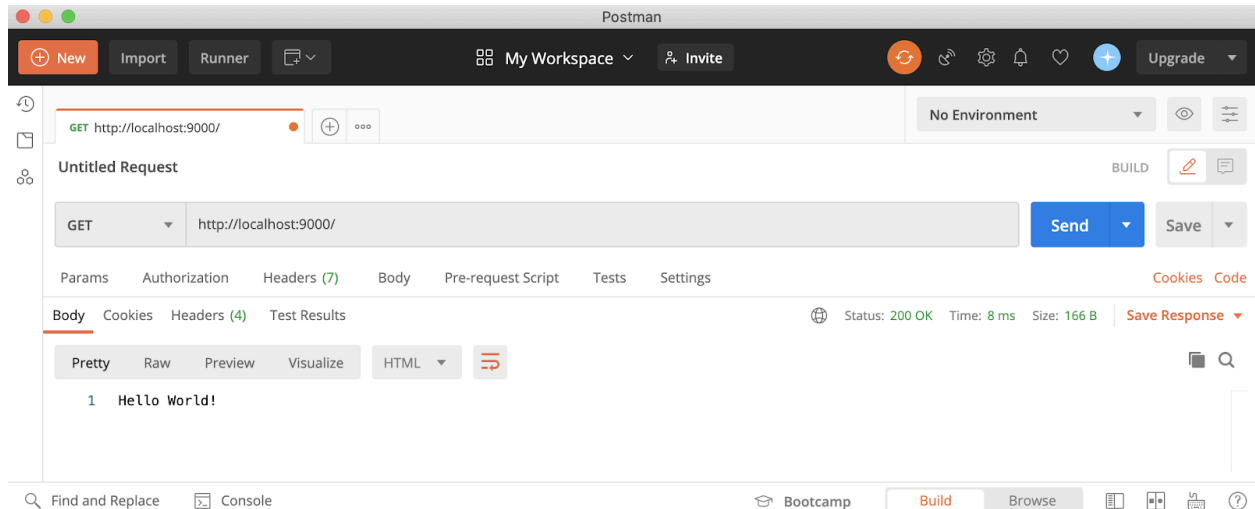
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello World!"

app.run(host="localhost", port=9000)
```

This is the most simple Flask application possible, that has a single callable endpoint at the empty path, which always returns the string *Hello World!*. Start the server by invoking the Python interpreter, and test the only endpoint with Postman!

```
$ python rest-server.py
* Serving Flask app "rest-server" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://localhost:9000/ (Press CTRL+C to quit)
127.0.0.1 - - [02/Sep/2020 12:37:13] "GET / HTTP/1.1" 200 -
```



As you can see, the returned HTTP response body is a plaintext string. In a REST API we typically use JSON as the data format in both directions: endpoints receive JSON in the request body of POST and PUT requests, and return JSON in the response.

To achieve this, first we need to make a JSON object from our simple string. Then, we should inform the caller that the response is not plaintext format, and send the “Content-Type” HTTP header, set to “application/json”. In Flask, if we want to send custom response headers, we also have to specify the HTTP response code, which in this case is simply 200 (Ok). Instead of just returning a string, we’ll need to return a tuple of (result string, response code, headers dictionary) as follows:

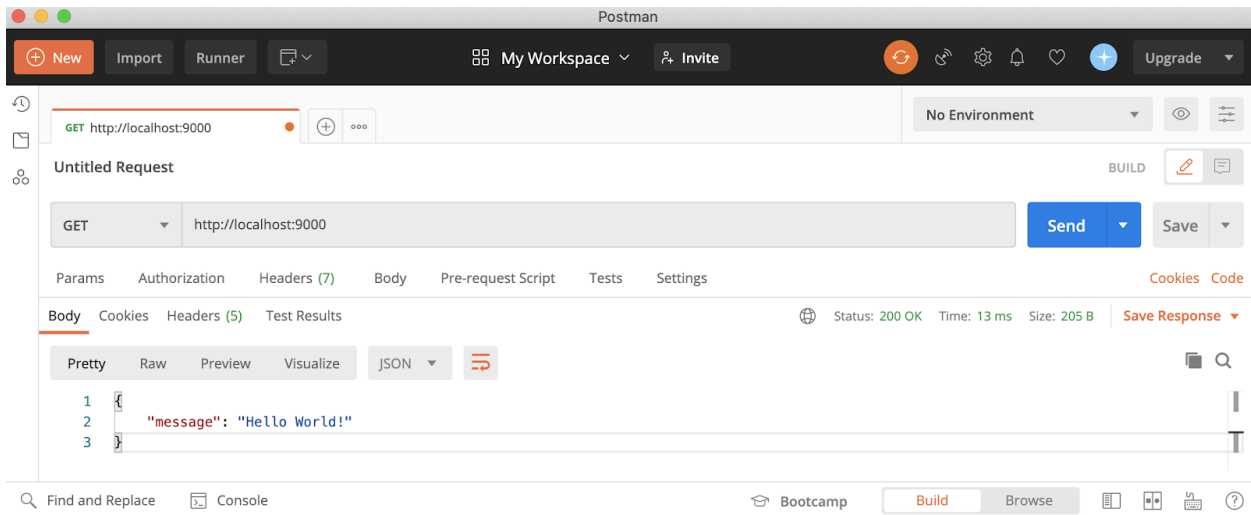
```
@app.route('/')
def hello():
    return {'message': 'Hello World!'}, 200, {'Content-Type': 'application/json'}
```

Actually, Flask has a built-in helper function called `make_response()`, which adds the appropriate Content-Type header.

```
from flask import Flask, make_response
...
@app.route('/')
def hello():
    return make_response({'message': 'Hello World!'})
```

Task 7: Update the hello function to return JSON and test it from Postman!

After changing the implementation of `hello()` to the code snippet above and restarting the server, Postman shows the new response with JSON syntax highlighting.



Using a database

Before adding more endpoints, let's see how to use a persistent database to store and retrieve metadata. At this point you need to make a design decision between relational or non-relational databases. In the lab we will use a very simple relational database called Sqlite3, but you can also use a different one like MySQL or a document datastore like MongoDB (which has an excellent [tutorial series](#)). The following tasks assume that you have Sqlite3 installed on your computer, which you can invoke with the `sqlite3` command.

Task 8: Create a new persistent Sqlite3 database and add a new table for the File entity.

Open the sqlite shell in the same folder where your `rest-server.py` script is, and pass "files.db" as an argument.

```
$ sqlite3 files.db
SQLite version 3.28.0 2019-04-15 14:49:49
Enter ".help" for usage hints.
sqlite>
```

This creates a new database file and you get an interactive shell to work on it. Copy and paste the following SQL command to create a table for our File entity.

```
CREATE TABLE `file` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT,
  `filename` TEXT,
  `size` INTEGER,
```

```
`content_type` TEXT,  
`created` DATETIME DEFAULT CURRENT_TIMESTAMP,  
`blob_name` TEXT  
);
```

When it comes to database connections, there are two strategies:

1. Open and close the connection for each database query separately.
2. Open the connection when a request arrives and close after it has been served

The second approach is preferable in our case, since our endpoints finish quickly (therefore they don't hold a connection from the pool without doing anything on it). Flask helps with this task in two ways: it provides the global `g` object, which is unique per request and can be accessed by multiple functions while serving the request. We will use it to store the database connection. Additionally, we can ask Flask to run a custom function after a request has been served. We will close the connection in this function.

```
from flask import Flask, make_response, g, request  
...  
  
def get_db():  
    # Connect to the sqlite DB at 'files.db' and store the connection in 'g.db'  
    # Re-use the connection if it already exists  
    if 'db' not in g:  
        g.db = sqlite3.connect(  
            'files.db',  
            detect_types=sqlite3.PARSE_DECLTYPES  
        )  
        # Enable casting Row objects to Python dictionaries  
        g.db.row_factory = sqlite3.Row  
  
    return g.db  
  
def close_db(e=None):  
    # Close the DB connection and remove it from the 'g' object  
    db = g.pop('db', None)  
  
    if db is not None:
```

```
db.close()

app = Flask(__name__)
# Close the DB connection after serving a request
app.teardown_appcontext(close_db)
```

In the endpoint functions all we need to do is get a reference to the DB connection when we need it (by calling `get_db()`). It is re-used and closed automatically.

We will use the database to implement a simple file storage REST API with the following endpoints:

- **POST /files**: Upload a new file. The file name, content type and data are encoded in the request body as a JSON object
- **GET /files**: List the stored files
- **GET /files/[id]**: Download the contents of the requested file
- **HEAD /files/[id]**: Return the metadata of the requested file
- **DELETE /files/[id]**: Remove a file

Task 9: Implement the upload file endpoint!

When a file is uploaded, its contents will be base64 encoded and included in the JSON request body, together with the file name and [type](#). The server endpoint decodes the binary file contents and saves it locally with a random generated name, using the `write_file()` helper function from earlier. It also calculates the size and stores a database record of the new file.

```
@app.route('/files', methods=['POST'])
def add_files():
    # Parse the request body as JSON and convert to a Python dictionary
    payload = request.get_json()
    filename = payload.get('filename')
    content_type = payload.get('content_type')
    # Decode the file contents and calculate its original size
    file_data = base64.b64decode(payload.get('contents_b64'))
    size = len(file_data)

    # Store the file locally with a random generated name
    blob_name = write_file(file_data)

    # Insert the File record in the DB
    db = get_db()
```

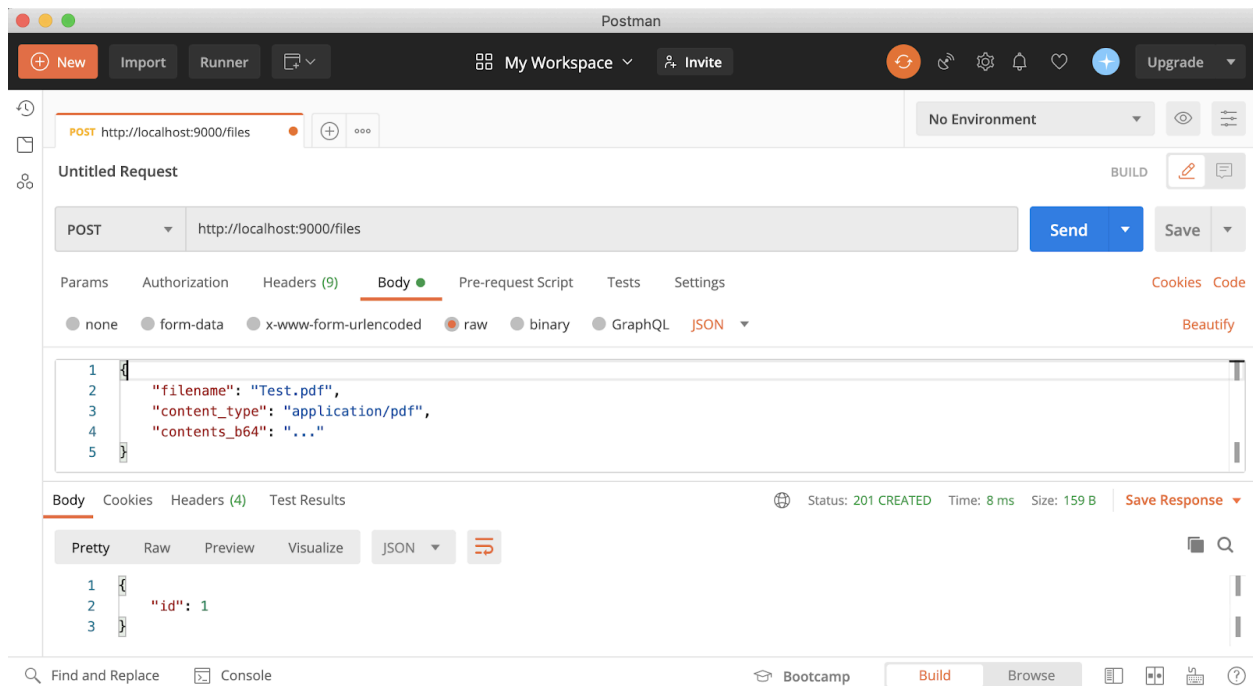
```

cursor = db.execute(
    "INSERT INTO `file`(`filename`, `size`, `content_type`, `blob_name`) VALUES
    (?, ?, ?, ?)",
    (filename, size, content_type, blob_name)
)
db.commit()

# Return the ID of the new file record with HTTP 201 (Created) status code
return make_response({"id": cursor.lastrowid }, 201)
#

```

Upload a file with Postman like you did with the RPC endpoint! Use the real content type of the file from [this list](#), and upload a file that can be displayed in a browser, so you can test the GET endpoint later (e.g. image, PDF, text file)!



Upload a few more files to the server! Also check what happens if you try to upload the same file multiple times.

Task 10: Implement the file list endpoint!

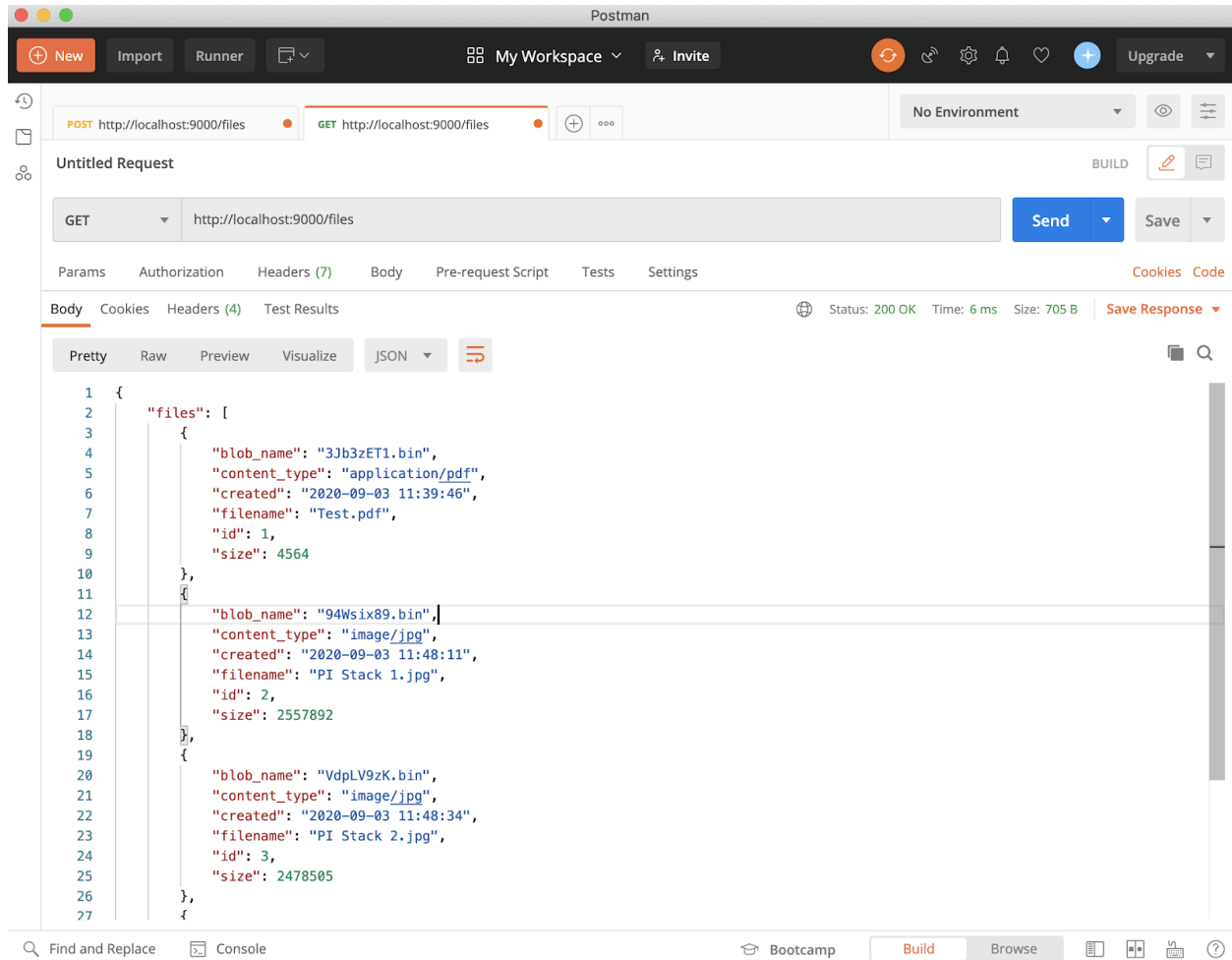
Simply query the database for every row of the file table, convert each Row instance to a Python dictionary and return it as a JSON object.

```
@app.route('/files', methods=['GET'])
def list_files():
    # Query the database for all files
    db = get_db()
    cursor = db.execute("SELECT * FROM `file`")
    if not cursor:
        return make_response({"message": "Error connecting to the database"}, 500)
    files = cursor.fetchall()

    # Convert files from sqlite3.Row object (which is not JSON-encodable) to
    # a standard Python dictionary simply by casting
    files = [dict(f) for f in files]

    return make_response({"files": files})
#
```

Restart the server and test the new endpoint from Postman!



Task 11: Implement the download endpoint!

This endpoint takes a File ID and returns the file contents with the appropriate content type, so the client (like a web browser) can display the file properly.

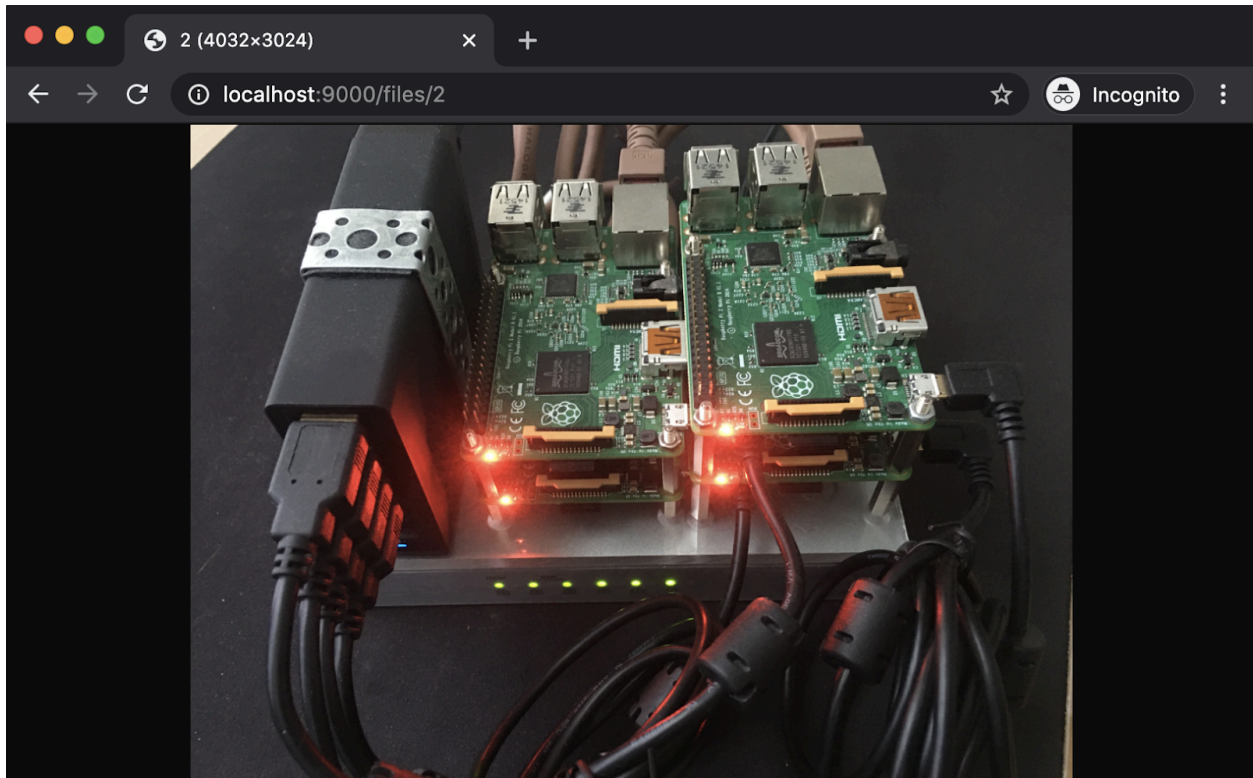
To accept a file ID as a GET parameter, we will define the endpoint URL with the pattern `/files/<int:file_id>`. As a result, Flask invokes the endpoint only if the request url has this structure, and the end is an integer. The endpoint function receives the parsed int as the `'file_id'` parameter. Additionally, Flask has a very handy helper function called `send_file()`, which takes care of constructing the response from a file path (or contents).

```
from flask import Flask, make_response, g, request, send_file
...
@app.route('/files/<int:file_id>', methods=['GET'])
def download_file(file_id):
    db = get_db()
    cursor = db.execute("SELECT * FROM `file` WHERE `id`=?", [file_id])
```

```
if not cursor:
    return make_response({"message": "Error connecting to the database"}, 500)

f = cursor.fetchone()
# Convert to a Python dictionary
f = dict(f)
print("File requested: {}".format(f))
# Return the binary file contents with the proper Content-Type header.
return send_file(f['blob_name'], mimetype=f['content_type'])
#
```

Test the endpoint from a browser!



Task 12 (Optional): Implement the get file metadata (HEAD) and file delete (DELETE) endpoints!

You have all the building blocks to write the HEAD and DELETE endpoints. Test them with Postman when you are done.